Titulo de nuestra tesis

Autores
Santiago Andrés Mesa Niño
Juan Camilo Carvajal Rodríguez
Andrés Felipe Manosalva Garzón
Juan Diego Arias Duran

Trabajo de grado para optar por el título de ingeniero de sistemas

Director
Rafael Vicente Paéz Méndez, PhD

Cliente
B.A.F: Blockchain Acceleration Foundation

Pontificia Universidad Javeriana
FACULTAD DE INGENIERÍA
Departamento de ingeniería de sistemas

Bogotá - Colombia, mayo 2025

- Rector de la Pontificia Universidad Javeriana: Padre Luis Fernando Múnera, S.J.
- Decano de la facultad de ingeniería: Diego Alejandro Patiño Guevara, Sc.D.
- Directora de la Carrera de ingeniería de sistemas: Andrea del Pilar Rueda Olarte, Ph.D.
- Director de proyecto: Rafael Vicente Paéz Méndez, PhD.

Artículo 23 de la resolución No. 13 de junio de 1946

"La universidad no se hace responsable de los conceptos emitidos por sus alumnos en sus proyectos de grado. Solo velará porque no se publique nada contrario al dogma y la moral católica y porque los trabajos no contengan ataques o polémicas puramente personales. Antes bien, que se vea en ellos el anhelo de buscar la verdad y la justicia".

Agradecimientos

Los autores desean expresar su más profundo agradecimiento a .

Queremos también agradecer a .

Extendemos nuestro agradecimiento a .

Finalmente, agradecemos a la cliente, por .

Índice

Página

1. Resumen **VI**

2. Abstract **VII**

3. Lista de Acrónimos y Definiciones **VIII**

4. Introducción **1**
4.1. Contexto del problema (ticketing digital, reventa y fraude) 1
4.2. Planteamiento investigativo (validación tecnológica) 1
4.3. Estructura del documento 1

5. Generalidades del proyecto **2**
5.1. Descripción del problema 2
5.2. Objetivos 2
5.2.1. Objetivo General 2
5.2.2. Objetivos Específicos 2
5.3. Alcance y delimitaciones 2
5.4. Metodología (investigación + desarrollo del prototipo) 3
5.5. Requerimientos 3
5.5.1. Requerimientos funcionales (mínimos: 10) 3
5.5.2. Requerimientos no funcionales (mínimos: 6, base ISO/IEC 27001) 4
5.5.3. Requerimientos complementarios (no obligatorios) 4
5.6. Estado del arte (comparación técnica y evidencia empírica) 5
5.7. Marco legal colombiano (caso de referencia: Ley 1493/2011 y fiscalización digital) 5

6. Marco Teórico **6**
6.1. Blockchain y contratos inteligentes 6
6.2. Red Stellar 6
6.2.1. Modelo de transacciones/operaciones y comisiones 7
6.2.2. Componentes de interacción (RPC, indexación de eventos) 7
6.3. Protocolo de Consenso Stellar (SCP) 7
6.3.1. Federated Byzantine Agreement: quórums y quorum slices (base teórica) 7
6.3.2. Propiedades: safety vs liveness (según documentación oficial) 8
6.3.3. Evidencia formal (pruebas/modelos sobre SCP) 8
6.4. Soroban (Rust/Wasm) 8
6.4.1. Storage y eventos 8
6.4.2. NFTs en Soroban 9
6.5. Tokens y pagos 9
6.5.1. XLM vía Stellar Asset Contract (SAC) 9
6.5.2. Token simulado USDC_SIM vía SAC 9

7. Arquitectura y diseño del sistema propuesto **10**
7.1. Actores, roles y límites de confianza (ticketera, vendedor, comprador, verificador) 10

7.2. Modelo del ticket NFT (unicidad, autenticidad y trazabilidad) 10
7.2.1. Identificador único y hash de metadatos (sin exponer datos sensibles) . . 11
7.2.2. Estados del ticket (VALIDO/USADO/CANCELADO) y patrón burn/remint 11
7.3. Diseño de contratos inteligentes (Soroban) 11
7.3.1. Contrato factory_contract 11
7.3.2. Contrato event_contract: funciones mint/list/buy/transfer/redeem . . . 12
7.3.3. Compra atómica (pago + comisiones + transferencia) 12
7.3.4. Política de comisiones (5 % organizador, 3 % plataforma) 13
7.3.5. Errores tipados y eventos on-chain 13
7.4. Arquitectura distribuida (3 nodos) 13
7.4.1. Nodo Web (UI ticketera) 13
7.4.2. Nodo API (Node.js) 13
7.4.3. Nodo Datos (PostgreSQL + Indexer) 14
7.5. Coleccionable Stellar Classic (NFT visible en billetera) 14
7.6. Indexación y auditoría (eventos on-chain hacia Postgres) 15
7.6.1. Ventana de retención de getEvents y necesidad de ingesta 15
7.7. Mapeo normativo simulado (Ley 1493/2011 en diseño, solo documental) 15

8. Implementación **16**
8.1. Implementación de contratos Soroban 16
8.1.1. NFT Ticket: emisión, propiedad y versionado 16
8.1.2. Marketplace y pagos (XLM vía SAC) 16
8.1.3. Redención y control de acceso 17
8.2. Backend Node.js + Stellar SDK 17
8.2.1. API (usuarios, eventos, carrito, checkout, transacciones) 17
8.2.2. Indexer (ingesta de eventos) 18
8.2.3. Persistencia PostgreSQL (historial, listados, auditoría) 18
8.3. Frontend web (simulación ticketera) 18
8.4. Despliegue distribuido (3 nodos) 19
8.5. Integración continua 19
8.6. Pruebas (unitarias, integración, antifraude, medición) 20

9. Estándares de ingeniería **20**
9.1. ISO/IEC 27001 (controles como requisitos no funcionales; sin certificación) . . . 20
9.2. ISO 9001 (gestión de calidad del SDLC y control documental) 20
9.3. Estándares de documentación, versionamiento y trazabilidad 20

10. Evaluación y resultados **21**
10.1. Metodología de medición (latencia E2E, confirmación, tasa de fallos) 21
10.2. Resultados del prototipo (Testnet) 21
10.3. Evaluación antifraude (doble redención, reventa múltiple, QR estático) 21
10.4. Discusión comparativa (contra estado del arte) 21

11. Conclusiones **22**
11.1. Conclusiones 22
11.2. Trabajo futuro (mainnet, KYC, integraciones, políticas avanzadas) 22

Referencias **23**

A. Anexos **23**
A.1. Manual de usuario 23
A.2. Manual de despliegue 23
A.3. Casos de prueba y scripts de medición 23
A.4. ABI/IDs de contratos y evidencias 23

Índice de figuras

Índice de tablas

1. Resumen

Esta tesis presenta el diseño y validación de una solución B2B de reventa de entradas basada en contratos inteligentes sobre Stellar. El enfoque del trabajo es investigativo y busca evaluar la aplicabilidad técnica del modelo para reducir fraude por duplicidad y reventa múltiple mediante unicidad del ticket, trazabilidad de transferencias y verificación criptográfica.

El sistema propuesto se implementa como un prototipo distribuido en testnet con tres nodos (web, API y datos), integrando contratos Soroban, indexación de eventos y persistencia off-chain para auditoría. Se define un modelo de ticket NFT con metadatos resumidos por hash, ciclo de vida de estados y reglas de compra atómica con comisiones parametrizadas.

La evaluación considera latencia extremo a extremo, tiempos de confirmación, tasa de fallos y escenarios antifraude. Los resultados permiten contrastar el prototipo frente a esquemas tradicionales de reventa y discutir sus implicaciones técnicas dentro del contexto académico.

Palabras clave: blockchain, Stellar, Soroban, contratos inteligentes, NFT, reventa de entradas.

2. Abstract

This thesis presents the design and validation of a B2B ticket resale solution based on smart contracts on Stellar. The research-oriented approach evaluates the technical applicability of the model to reduce fraud related to ticket duplication and multiple resale through ticket uniqueness, transfer traceability, and cryptographic verification.

The proposed system is implemented as a distributed prototype on testnet with three nodes (web, API, and data), integrating Soroban contracts, event indexing, and off-chain persistence for auditing. An NFT ticket model is defined with metadata integrity through hashing, lifecycle states, and atomic purchase rules with parameterized commissions.

The evaluation includes end-to-end latency, confirmation times, failure rate, and anti-fraud scenarios. The results enable comparison against traditional resale schemes and support technical discussion in an academic setting.

Keywords: blockchain, Stellar, Soroban, smart contracts, NFT, ticket resale.

3. Lista de Acrónimos y Definiciones

- **API:** Application Programming Interface.
- **B2B:** Business to Business.
- **E2E:** End to End.
- **FBA:** Federated Byzantine Agreement.
- **NFT:** Non-Fungible Token.
- **RPC:** Remote Procedure Call.
- **SAC:** Stellar Asset Contract.
- **SCP:** Stellar Consensus Protocol.
- **SGSI:** Sistema de Gestión de Seguridad de la Información.
- **TPS:** Transactions per Second.
- **USDC_SIM:** Token de prueba para simular pagos tipo stablecoin en testnet.

4. Introducción

4.1. Contexto del problema (ticketing digital, reventa y fraude)

La digitalización de la industria de boletería ha facilitado la compra y transferencia de entradas, pero también ha incrementado riesgos de fraude en mercados secundarios. Entre los casos más frecuentes se encuentran la duplicidad de códigos de acceso y la reventa múltiple de un mismo activo, lo que afecta la confianza de compradores, organizadores y plataformas de ticketing.

4.2. Planteamiento investigativo (validación tecnológica)

Este trabajo aborda el problema desde una perspectiva de validación tecnológica aplicada. La investigación propone y evalúa un prototipo B2B basado en contratos inteligentes sobre Stellar para modelar unicidad, autenticidad y trazabilidad del ticket digital durante el flujo de reventa.

4.3. Estructura del documento

El documento se organiza en ocho secciones principales: generalidades del proyecto, marco teórico, arquitectura y diseño, implementación, estándares de ingeniería, evaluación y resultados, conclusiones, y anexos. Esta secuencia permite conectar el fundamento conceptual con la validación experimental del prototipo.

5. Generalidades del proyecto

5.1. Descripción del problema

La reventa de entradas en canales digitales presenta tres problemas técnicos verificables: (i) duplicidad de códigos QR estáticos, que permite el ingreso múltiple con un mismo identificador; (ii) reventa concurrente del mismo activo a varios compradores en plataformas que no garantizan unicidad transaccional; y (iii) ausencia de trazabilidad pública de la cadena de propiedad, lo que impide al comprador final verificar el origen de la entrada antes de su uso.

Adicionalmente, los canales tradicionales de liquidación entre vendedor, plataforma y organizador operan sobre infraestructura bancaria con tiempos de finalidad entre 72 y 120 horas hábiles, frente a los 3 a 5 segundos que ofrece la red Stellar, lo que genera una asimetría entre la entrega del activo digital y la confirmación efectiva del pago. Esta asimetría es uno de los vectores explotados en escenarios de fraude documentados en mercados secundarios.

5.2. Objetivos

5.2.1. Objetivo General

Desarrollar una solución B2B de reventa de tickets, basada en la tecnología de contratos inteligentes de Stellar, que garantice la unicidad y autenticidad de cada activo digital, reduciendo al mínimo el fraude y la especulación en la reventa de entradas para eventos de entretenimiento.

5.2.2. Objetivos Específicos

- Crear un sistema de reventa B2B de tickets basado en contratos inteligentes y NFTs que aseguren autenticidad, originalidad y trazabilidad.
- Desarrollar un contrato inteligente que genere tickets como NFTs, permita la transferencia de propiedad y automatice la distribución de comisiones.
- Implementar un prototipo que simule el proceso completo de reventa en la red de prueba de Stellar, desde la venta hasta la compra.
- Gestionar el funcionamiento de la herramienta a través de una implementación web que permita la simulación y visualización del modelo y cómo este se incorpora al modelo de Blockchain.

5.3. Alcance y delimitaciones

El alcance del trabajo se limita al mercado secundario de entradas para eventos de entretenimiento, con despliegue del prototipo sobre la red de prueba (testnet) de Stellar. El sistema cubre el ciclo completo del ticket: emisión por parte del organizador, venta primaria, listado de reventa, compra atómica con distribución de comisiones, transferencia de propiedad, redención en puerta y cancelación.

Quedan fuera de alcance, para esta versión del prototipo: integración con pasarelas de pago fiat reales, procesos productivos de Know Your Customer (KYC), despliegue definitivo en mainnet, acoplamiento con plataformas comerciales de ticketing de terceros, y certificación formal bajo

ISO/IEC 27001 o ISO 9001 (los estándares se aplican como marco de referencia metodológico, no como objeto de auditoría).

5.4. Metodología (investigación + desarrollo del prototipo)

La metodología combina cuatro fases articuladas: (i) revisión técnico-científica de Stellar, SCP, Soroban y patrones de NFT, sustentada en documentación oficial y literatura indexada; (ii) diseño de arquitectura del prototipo, con definición de actores, contratos y separación de responsabilidades on-chain/off-chain; (iii) implementación incremental por fases, donde cada fase entrega un componente funcional verificable (contratos, backend, frontend, indexador); y (iv) evaluación experimental sobre testnet con métricas reproducibles de latencia, costo y comportamiento antifraude.

Esta metodología se enmarca en un ciclo de vida del desarrollo de software (SDLC) compatible con los principios de ISO 9001 en lo que respecta a control de versiones, trazabilidad de cambios y registro documental de cada iteración. El repositorio público del proyecto (`https://github.com/Tesis-Stellar/stellar-tickets`) y su pipeline de integración continua (GitHub Actions sobre el contrato Soroban) constituyen la evidencia operativa de esta práctica.

5.5. Requerimientos

5.5.1. Requerimientos funcionales (mínimos: 10)

Se definen los siguientes diez requerimientos funcionales, todos verificables sobre el prototipo desplegado en testnet:

- **RF-01 — Emisión de boletos como NFT:** el contrato debe permitir al organizador autenticado emitir un boleto identificado por una tupla (`ticket_root_id`, `version`) única en el contrato del evento.
- **RF-02 — Listado en reventa:** el propietario actual de un boleto debe poder listar el activo en venta secundaria definiendo un precio en stroops.
- **RF-03 — Cancelación de listado:** el propietario debe poder retirar el boleto del mercado secundario antes de su compra.
- **RF-04 — Compra primaria:** el sistema debe permitir la compra inicial del boleto al organizador, transfiriendo el 100 % del precio al organizador.
- **RF-05 — Compra de reventa atómica:** la compra en mercado secundario debe ejecutar en una sola transacción el pago al vendedor, la distribución de comisiones al organizador y la plataforma, y la transferencia de propiedad.
- **RF-06 — Distribución automática de comisiones:** el contrato debe aplicar las comisiones parametrizadas (5 % organizador, 3 % plataforma en la configuración del prototipo) sobre cada operación de reventa.
- **RF-07 — Invalidación del ticket previo (burn/remint):** en cada reventa, la versión anterior del boleto debe quedar marcada como inválida y emitirse una nueva versión con `version+1` a nombre del comprador.

- **RF-08 — Verificación y redención en puerta:** solo direcciones registradas como verificadores autorizados pueden marcar un boleto como redimido (`usado`).
- **RF-09 — Trazabilidad de eventos on-chain:** el contrato debe emitir eventos para cada operación crítica (creación, listado, cancelación, compra primaria, reventa, redención e invalidación), de modo que un indexador externo pueda reconstruir el historial completo.
- **RF-10 — Consulta de estado y propiedad:** el contrato debe exponer funciones de consulta que permitan verificar el propietario actual, la versión vigente y el ciclo de vida de cualquier boleto sin requerir privilegios especiales.

5.5.2. Requerimientos no funcionales (mínimos: 6, base ISO/IEC 27001)

Se establecen seis requerimientos no funcionales tomando como marco de referencia los controles del Anexo A de ISO/IEC 27001:2022:

- **RNF-01 — Control de acceso (A.5.15, A.8.3):** las operaciones que modifican el estado del contrato (`crear_boleto`, `listar_boleto`, `cancelar_venta`, `comprar_boleto`, `redimir_boleto`, `invalidar_boleto`) deben ejecutar `require_auth()` sobre el rol correspondiente (organizador, propietario, comprador o verificador).
- **RNF-02 — Integridad criptográfica del registro (A.8.24):** cada transacción debe quedar registrada en el ledger con firma digital y referencia inmutable al hash del bloque anterior, de manera que cualquier alteración posterior sea detectable.
- **RNF-03 — Trazabilidad y registro de eventos (A.8.15):** todas las transiciones de estado del boleto deben emitir eventos on-chain consumibles por el indexador, con persistencia off-chain en PostgreSQL para historial de largo plazo.
- **RNF-04 — Gestión de claves criptográficas (A.8.24):** las llaves privadas operacionales (organizador, plataforma, verificador) deben permanecer fuera del repositorio del código y del frontend; el comprador firma sus propias transacciones desde una billetera personal (Freighter) sin que la llave salga del entorno del usuario.
- **RNF-05 — Disponibilidad operativa (A.5.30, A.8.14):** el backend debe operar en arquitectura sin estado, replicable horizontalmente, con base de datos relacional gestionada externamente; el indexador debe recuperarse automáticamente de errores transitorios y de ventanas de retención del RPC.
- **RNF-06 — Validación y manejo de excepciones (A.8.26, A.8.29):** el contrato debe definir un conjunto cerrado de errores tipados (16 códigos en `ErrorContrato`) y rechazar toda operación que viole las precondiciones, sin estados parciales.

5.5.3. Requerimientos complementarios (no obligatorios)

Como extensiones funcionales no obligatorias, el sistema contempla: tope de precio máximo de reventa configurable por evento, listas de bloqueo de direcciones, soporte para activos de pago adicionales (USDC_SIM como caso de prueba), políticas de cancelación con devolución parcial, y panel de analítica para detección de patrones de especulación.

5.6. Estado del arte (comparación técnica y evidencia empírica)

El contraste entre los esquemas tradicionales de reventa y un esquema basado en contratos inteligentes sobre Stellar se resume en los siguientes parámetros técnicos:

- **Tiempo de liquidación:** Stellar opera con tiempos de finalidad entre 3 y 5 segundos por transacción; las pasarelas tradicionales y la banca corresponsal liquidan transferencias internacionales en ventanas de 72 a 120 horas hábiles.
- **Costo nominal por operación:** Stellar aplica una tarifa base de 100 stroops por operación (0,00001 XLM); las pasarelas tradicionales aplican comisiones del 1,5 % al 3,5 % sobre el monto, sumadas a una tarifa fija.
- **Capacidad de procesamiento:** Stellar reporta capacidad de 1 000 a 3 000 TPS, frente al rango de 15 a 30 TPS de Ethereum 1.0, lo que define una diferencia de capacidad teórica del orden de 100x.
- **Mecanismo antifraude:** la unicidad del NFT (tupla `ticket_root_id`, `version`) y el patrón de *burn/remint* en cada reventa eliminan la posibilidad de coexistencia de dos copias válidas del mismo activo, problema que sí ocurre en los códigos QR estáticos no anclados a un registro distribuido.
- **Trazabilidad pública:** todo evento on-chain queda registrado de forma verificable mediante exploradores públicos (*Stellar Expert*), mientras que en los mercados secundarios tradicionales el historial de transferencias no es accesible al comprador final.

5.7. Marco legal colombiano (caso de referencia: Ley 1493/2011 y fiscalización digital)

La Ley 1493 de 2011 regula los espectáculos públicos de las artes escénicas en Colombia y establece, entre otros mecanismos, una contribución parafiscal del 10 % sobre el valor de la boletería de los espectáculos cuyo precio individual sea igual o superior a tres Unidades de Valor Tributario (UVT). Esta contribución debe ser recaudada por el productor del espectáculo y reportada a la autoridad competente, lo que impone requisitos de trazabilidad sobre cada boleto vendido y su precio efectivo de transacción.

En el modelo propuesto, el contrato inteligente automatiza el cálculo de esta contribución en el punto exacto de la venta primaria y de cada reventa, dado que: (i) el precio efectivo queda registrado on-chain como parámetro de la operación de compra; (ii) cada cambio de propiedad emite un evento (`boleto_comprado_primario`, `boleto_revendido`) con el precio asociado; y (iii) el indexador persiste estos eventos en una base de datos relacional auditable. Esta arquitectura facilita el cumplimiento del deber de reporte de la ticketera, dado que el cálculo del 10 % sobre cada operación que supere el umbral legal se obtiene como una agregación determinística sobre el historial indexado, sin depender de reportes manuales ni de conciliaciones a posteriori.

El presente trabajo utiliza la Ley 1493/2011 como caso de referencia académica para discutir la fiscalización digital en el contexto del prototipo. La discusión jurídica se modela como mapeo normativo dentro del diseño y no constituye una declaración de cumplimiento regulatorio productivo.

6. Marco Teórico

6.1. Blockchain y contratos inteligentes

Una cadena de bloques es una estructura de datos distribuida, ordenada y enlazada criptográficamente, en la que un conjunto de nodos mantiene una réplica del registro y aplica un protocolo de consenso para acordar el orden y validez de las transacciones. Cada bloque contiene un encabezado con la referencia al hash del bloque anterior, una raíz de Merkle de las transacciones y metadatos del consenso, lo que permite la detección determinística de cualquier alteración posterior al registro.

Sobre esta infraestructura, los contratos inteligentes son programas con estado persistente y ejecución determinística cuyas transiciones se validan en cada nodo bajo el mismo conjunto de reglas. En el contexto de este trabajo, los contratos inteligentes se utilizan como mecanismo de coordinación entre la ticketera, los compradores y la plataforma de reventa, transfiriendo al protocolo la verificación de propiedad, la unicidad del activo y la distribución de comisiones, sin requerir confianza unilateral en un intermediario.

6.2. Red Stellar

Stellar es una red pública orientada a la transferencia de valor digital y a la emisión de activos tokenizados. La red opera bajo el Stellar Consensus Protocol (SCP) e integra desde el protocolo 20 una capa de contratos inteligentes denominada Soroban. Sus parámetros operacionales relevantes para este trabajo son los siguientes:

- **Capacidad de procesamiento:** la documentación oficial reporta una capacidad de entre 1 000 y 3 000 transacciones por segundo (TPS) dependiendo del tipo de operación. Como punto de comparación, Ethereum en su modelo previo a la transición a Proof of Stake operaba en el rango de 15 a 30 TPS, lo que implica una diferencia de capacidad teórica del orden de 100x a favor de Stellar.
- **Tiempo de liquidación:** cada cierre de ledger en Stellar ocurre en un intervalo de aproximadamente 5 segundos, lo que define el tiempo de finalidad de una transacción entre 3 y 5 segundos. En contraste, las transferencias bancarias internacionales y las liquidaciones de pasarelas tradicionales se ejecutan en ventanas de 72 a 120 horas hábiles.
- **Costo por operación:** el costo base por operación en Stellar es de 100 stroops, equivalentes a 0,00001 XLM. Las pasarelas de pago tradicionales aplican comisiones porcentuales en el rango del 1,5 % al 3,5 % sobre el monto, sumadas a una tarifa fija por transacción.
- **Eficiencia energética:** el SCP no utiliza minería ni Proof of Work; las decisiones de consenso se alcanzan mediante intercambio de mensajes firmados entre nodos, lo que elimina el costo energético asociado al cómputo intensivo de redes basadas en PoW.

Estos parámetros se utilizan en el presente trabajo como referencia objetiva para la evaluación del prototipo en testnet y para el contraste con esquemas tradicionales de reventa.

6.2.1. Modelo de transacciones/operaciones y comisiones

Stellar diferencia los conceptos de *transacción* y *operación*. Una transacción es la unidad atómica firmada y enviada al ledger; cada transacción puede contener hasta 100 operaciones, todas las cuales se aplican en un único cierre de ledger o ninguna se aplica en caso de fallo. Esta semántica de *todo o nada* es relevante para el diseño de la compra de reventa, en la cual el pago al vendedor, la distribución de comisiones al organizador y a la plataforma, y la transferencia de propiedad del ticket deben ejecutarse de manera indivisible.

La comisión por transacción se calcula como *base fee* (100 stroops por defecto) multiplicada por el número de operaciones contenidas. En condiciones de saturación del ledger, la red aplica un mecanismo de *surge pricing* que selecciona transacciones según la comisión ofertada, lo que constituye el único canal por el cual el costo nominal puede incrementarse durante la operación.

6.2.2. Componentes de interacción (RPC, indexación de eventos)

La interacción entre aplicaciones cliente y la red se realiza a través de dos componentes principales: *Horizon*, que expone operaciones clásicas (pagos, change_trust, manejo de activos), y *Soroban RPC*, que expone simulación, envío y consulta de transacciones de contratos inteligentes. Ambas interfaces operan sobre HTTP/JSON-RPC y se invocan desde el SDK oficial `@stellar/stellar-sdk`.

Soroban RPC ofrece consulta de eventos emitidos por contratos a través del método `getEvents`. Esta consulta tiene una ventana de retención limitada, lo que obliga a las aplicaciones que requieren historial completo a implementar un componente indexador que consuma eventos en intervalos cortos y los persista en una base de datos off-chain. En el prototipo, este patrón es implementado por el módulo `indexer` del backend, que consulta el RPC cada 5 segundos y procesa los siete tipos de eventos definidos por el contrato (`boleto_creado`, `boleto_listado`, `venta_cancelada`, `boleto_comprado_primario`, `boleto_revendido`, `boleto_redimido`, `boleto_invalidado_ev`

6.3. Protocolo de Consenso Stellar (SCP)

El SCP es una instancia del paradigma de Federated Byzantine Agreement (FBA) en la que el conjunto de nodos participantes y la composición de los quórums no están definidos por una autoridad central, sino que cada nodo declara su propia configuración de confianza. Esta característica diferencia al SCP de los algoritmos clásicos de consenso bizantino con membresía fija, así como de los protocolos de Proof of Work y Proof of Stake.

6.3.1. Federated Byzantine Agreement: quórums y quorum slices (base teórica)

En el modelo FBA, cada nodo v define un conjunto de *quorum slices*, donde cada slice es un subconjunto de nodos cuya colaboración es suficiente para que v acepte un valor como acordado. Un *quórum* es un conjunto de nodos que contiene al menos un quorum slice de cada uno de sus miembros. La propiedad clave del modelo es la *intersección de quórums*: si todo par de quórums comparte al menos un nodo correcto, el sistema mantiene safety incluso ante nodos bizantinos, sin requerir un acuerdo global previo sobre la membresía total.

6.3.2. Propiedades: safety vs liveness (según documentación oficial)

El SCP distingue dos propiedades fundamentales del consenso:

- **Safety:** ningún par de nodos correctos puede acordar valores incompatibles para el mismo slot del ledger. Esta propiedad se preserva siempre que se cumpla la intersección de quórums entre nodos correctos.
- **Liveness:** los nodos correctos eventualmente acuerdan un valor para cada slot. Esta propiedad puede degradarse temporalmente bajo particiones de red o configuraciones de quórum desconectadas, pero no compromete la consistencia ya alcanzada.

El diseño prioriza safety sobre liveness, en línea con el caso de uso de transferencia de valor: ante un escenario adverso, el SCP detiene el avance del ledger en lugar de producir bifurcaciones. Para el problema de reventa de entradas, esta priorización es relevante porque garantiza que ningún ticket se pueda asignar simultáneamente a dos compradores distintos en estados acordados como finales.

6.3.3. Evidencia formal (pruebas/modelos sobre SCP)

El whitepaper de Mazières "The Stellar Consensus Protocol: A Federated Model for Internet-level Consensus" presenta las pruebas de safety bajo intersección de quórums y de liveness bajo supuestos de red asíncrona pero correcta. Trabajos posteriores han modelado el protocolo en lógicas formales y herramientas de model checking, validando las propiedades originales y caracterizando el comportamiento bajo configuraciones de confianza específicas. Estos resultados sustentan la utilización del SCP como infraestructura de consenso en el presente trabajo.

6.4. Soroban (Rust/Wasm)

Soroban es la plataforma de contratos inteligentes de Stellar. Sus contratos se escriben en Rust contra el crate `soroban-sdk` (versión 23 en este trabajo) y se compilan al target `wasm32v1-none`, generando un módulo WebAssembly que se sube al ledger y se invoca desde transacciones. El modelo de ejecución es determinístico, sin acceso a fuentes no reproducibles (relojes locales, aleatoriedad no derivada del ledger, syscalls del sistema operativo), lo que garantiza que toda invocación produzca el mismo resultado en cualquier nodo.

Para reducir el tamaño del binario y los costos de despliegue, el perfil de compilación `release` configurado en este proyecto utiliza `opt-level="z"`, *Link-Time Optimization* habilitado, símbolos eliminados (`strip = true`) y manejo de pánico mediante `abort`.

6.4.1. Storage y eventos

Los contratos Soroban gestionan tres tipos de almacenamiento: `Persistent` (estado de larga duración con costo de *rent* cada cierto número de ledgers), `Temporary` (estado de corto plazo con costo reducido) e `Instance` (estado asociado al binario del contrato). El uso del tipo apropiado define el costo de *rent* de cada entrada, lo que constituye una decisión de diseño con impacto directo en los costos operacionales.

La emisión de eventos se realiza mediante el método `events().publish()` sobre el entorno del contrato. Cada evento se compone de *topics* indexables y un *body* con el cuerpo serializable.

Los eventos no se almacenan en el storage del contrato sino en el meta del ledger, lo que permite a los indexadores consumirlos sin afectar el costo de *rent* del contrato. En el contrato `event_contract` de este trabajo se definen siete eventos que cubren el ciclo de vida completo del boleto, lo que habilita la sincronización determinística entre el estado on-chain y la base de datos relacional off-chain.

6.4.2. NFTs en Soroban

El estándar NFT en Soroban define una representación de activos no fungibles caracterizada por un identificador único, un propietario (de tipo `Address`) y reglas de transferencia controladas por el contrato emisor. La unicidad del identificador garantiza la imposibilidad de duplicar un mismo activo, lo que se utiliza en el presente trabajo como mecanismo principal de prevención de duplicidad de códigos QR en la reventa.

El contrato `event_contract` implementa un patrón de NFT extendido con versionado: cada boleto se identifica por la tupla (`ticket_root_id`, `version`), donde `ticket_root_id` es estable a lo largo de toda la vida del activo y `version` se incrementa en cada operación de reventa mediante un esquema de *burn* de la versión anterior y *remint* de una versión nueva. Este patrón conserva trazabilidad completa del historial sin reutilizar identificadores de propietarios anteriores.

6.5. Tokens y pagos

6.5.1. XLM vía Stellar Asset Contract (SAC)

XLM es el activo nativo de la red Stellar. A través del Stellar Asset Contract (SAC), XLM y cualquier activo emitido en Stellar se exponen como tokens compatibles con la interfaz estándar de Soroban (`transfer`, `balance`, `approve`). El contrato SAC del XLM en testnet utilizado en este trabajo es `CDLZFC3SYJYDZT7K67VZ75HPJVIEUVNIXF47ZG2FB2RMQQVU2HHGCYSC`. Esta integración permite que las operaciones de pago de la compra de reventa se ejecuten dentro del mismo flujo de invocación del contrato `event_contract`, sin requerir transacciones separadas para el pago y la transferencia de propiedad.

6.5.2. Token simulado USDC_SIM vía SAC

Como extensión experimental se contempla la emisión de un activo de prueba denominado USDC_SIM, expuesto también vía SAC, para evaluar el comportamiento del flujo de reventa con un token de referencia estable. El uso de USDC_SIM se limita al entorno de testnet y no implica integración con el emisor real de USDC en mainnet.

7. Arquitectura y diseño del sistema propuesto

7.1. Actores, roles y límites de confianza (ticketera, vendedor, comprador, verificador)

El sistema define cinco actores principales, cada uno con un rol acotado y verificado por el contrato a través de `require_auth()` sobre la dirección Stellar correspondiente:

- **Administrador del Factory:** dirección autorizada a desplegar nuevos contratos de evento mediante el contrato `factory_contract`. En el prototipo desplegado en testnet la dirección utilizada es `GBM6N2SUCK3Y6I5DHQKULZD3W27EYMU37VYHNKWLVBNS6VYZHRJPWJBT`.
- **Organizador del evento (ticketera B2B):** dirección registrada como organizador en el contrato del evento. Es la única autorizada a invocar `crear_boleto` y a registrar verificadores. En testnet: `GC2X5T6HTLJJ7P2EASP4PK64ZD24KDN57F6B4XEDA3AF4YX5PTGQKJSS`.
- **Plataforma:** dirección receptora de la comisión de plataforma en cada operación de reventa. En testnet: `GBMFIYOGHHNXJGUVWXTDUMLZJ2IRO3T2OOPG5CQPBEVLA2OO3SYPK2B2`.
- **Verificador en puerta:** dirección autorizada por el organizador para invocar `redimir_boleto` en el momento del ingreso al evento. En testnet: `GBJOI4X22GZY26YZASKHV7ZWYEZVU7ZN77EXZFE7WOCMAGQHLR26EMBZ2`.
- **Vendedor y comprador:** usuarios finales del mercado secundario. El propietario actual firma las operaciones `listar_boleto` y `cancelar_venta`; el comprador firma `comprar_boleto` desde su billetera personal (Freighter) sin que la llave privada salga del navegador.

Los límites de confianza separan dos planos: el plano on-chain, en el que los actores actúan exclusivamente a través de firmas digitales validadas por el SCP, y el plano off-chain, conformado por el backend, el indexador y la base de datos relacional. El backend opera bajo un modelo *stateless XDR proxy*: construye transacciones sin firmar (XDR) cuando el firmante es el comprador, y firma con la llave del organizador únicamente las operaciones que corresponden a su rol (`crear_boleto`, `listar_boleto`, `cancelar_venta`). En ningún momento el backend almacena llaves privadas de usuarios finales.

7.2. Modelo del ticket NFT (unicidad, autenticidad y trazabilidad)

Cada boleto se modela en el contrato `event_contract` mediante la estructura `Boleto`, con los siguientes campos: `ticket_root_id` (identificador estable, `u32`), `version` (incrementable en reventa, `u32`), `id_evento` (`u32`), `propietario` (`Address`), `precio` (`i128` en stroops), `en_venta` (`bool`), `es_reventa` (`bool`), `usado` (`bool`) e `invalidado` (`bool`).

La unicidad se garantiza por la clave de almacenamiento `Boleto(ticket_root_id, version)` y por el contador `ContadorBoletos` que asigna identificadores monotónicamente crecientes en la operación de emisión. La autenticidad se valida mediante la firma del organizador en el momento de la emisión y por la firma del propietario en cada operación posterior. La trazabilidad queda garantizada por la emisión de eventos on-chain que el indexador persiste en PostgreSQL para análisis histórico.

7.2.1. Identificador único y hash de metadatos (sin exponer datos sensibles)

La separación de datos sensibles y operacionales se implementa de la siguiente manera: el contrato almacena exclusivamente datos operacionales públicos (identificadores, propietario, precio, estados booleanos); los datos personales del comprador (nombre, documento, correo) se almacenan en la base de datos relacional off-chain bajo el esquema `ticketing`, asociados al usuario por su `user_id` y nunca por su llave privada. La vinculación entre el plano on-chain y el plano off-chain se realiza por la dirección Stellar (`wallet_address`) que el usuario asocia voluntariamente a su cuenta en el momento de conectar Freighter.

7.2.2. Estados del ticket (VALIDO/USADO/CANCELADO) y patrón burn/remint

El ciclo de vida del boleto se modela mediante los estados `ACTIVE` (`usado=false`, `invalidado=false`), `USED` (`usado=true`) y `CANCELLED` (`invalidado=true`). El estado `ACTIVE` admite operaciones de listado, cancelación de listado y compra; el estado `USED` se alcanza por invocación de `redimir_boleto` por parte de un verificador autorizado; el estado `CANCELLED` se alcanza por invocación de `invalidar_boleto` por parte del organizador.

En cada operación de reventa el contrato aplica el patrón *burn/remint*: la versión vigente se marca como `invalidado=true` y se almacena bajo `Boleto(ticket_root_id, version)`; simultáneamente se inserta una nueva entrada `Boleto(ticket_root_id, version+1)` con el comprador como propietario, y se actualiza el índice `VersionActual(ticket_root_id)` para que las consultas de propiedad apunten a la nueva versión. Este patrón conserva el historial completo de propietarios anteriores y elimina la posibilidad de coexistencia de dos copias válidas del mismo activo.

7.3. Diseño de contratos inteligentes (Soroban)

El sistema utiliza dos contratos Soroban con responsabilidades acotadas: `factory_contract` y `event_contract`. Esta separación responde al principio de aislamiento de fallos: el estado de cada evento queda confinado en su propio contrato, lo que evita que una falla en un evento afecte la operación de los demás.

7.3.1. Contrato factory_contract

El contrato `factory_contract` implementa el patrón Factory para el despliegue de un `event_contract` independiente por cada evento. Su estructura `ConfiguracionEvento` agrupa los parámetros del despliegue: `id_evento`, `organizador`, `token_pago`, `comision_organizador` (`u32`), `comision_plataforma` (`u32`), `wallet_organizador`, `wallet_plataforma` y `capacidad_total`. La función `crear_evento_contrato` requiere doble autenticación (administrador del factory y organizador del evento) y emite el evento `EventoCreado(id, organizador, contrato, capacidad)`.

El despliegue en producción se realiza mediante `deployer().with_current_contract(salt).deploy_v2(hash, ())`, donde `salt` se compone de `id_evento` (4 bytes en *big endian*) seguido de relleno en cero. El hash del módulo WASM utilizado en testnet es `32fbb9bf4e7f803c1e5caf3c2744e1efedb5197 5fd2775ebdfe1475e999bb0ef`. Al momento del cierre de esta tesis, el factory tiene desplegados 11 contratos de evento sobre testnet.

7.3.2. Contrato event_contract: funciones mint/list/buy/transfer/redeem

El contrato `event_contract` expone el siguiente conjunto funcional, con todas las identidades en español por trazabilidad académica:

- `inicializar(organizador, plataforma, token_pago, comision_org, comision_plat)`: configuración única, rechazada con error `YaInicializado` (1) si se invoca dos veces.
- `crear_boleto(id_evento, precio) ->u32`: emite un boleto con `version=1` y devuelve el `ticket_root_id`. Solo el organizador puede invocarla.
- `listar_boleto(ticket_root_id, nuevo_precio)` y `cancelar_venta(ticket_root_id)`: requieren firma del propietario actual; rechazadas si el boleto está `usado` (6) o `invalidado` (13).
- `comprar_boleto(ticket_root_id, comprador) ->u32`: aplica dos flujos diferenciados según el valor de `es_reventa` (ver subsección siguiente).
- `agregar_verificador / remover_verificador / es_verificador`: gestión del rol verificador por parte del organizador.
- `redimir_boleto(ticket_root_id, verificador)`: solo direcciones registradas como verificadoras.
- `invalidar_boleto(ticket_root_id)`: cancelación administrativa por parte del organizador.
- Consultas de solo lectura: `obtener_boleto`, `obtener_boleto_version`, `obtener_version_vigente`, `obtener_propietario`, `obtener_boletos_reventa`, `obtener_boletos_evento`.

7.3.3. Compra atómica (pago + comisiones + transferencia)

La función `comprar_boleto` aplica dos flujos según el estado del boleto:

- **Venta primaria** (`es_reventa=false`): el 100 % del precio se transfiere a la dirección del organizador. La operación actualiza el registro existente in-place y marca `es_reventa=true` para todas las operaciones futuras del mismo `ticket_root_id`.
- **Reventa** (`es_reventa=true`): el contrato calcula tres montos a partir del precio listado: `comision_organizador` (5 %), `comision_plataforma` (3 %) y `monto_vendedor` (92 %). Las tres transferencias y el cambio de propiedad se ejecutan en la misma invocación; si cualquier paso falla, la transacción completa se revierte por la semántica atómica del ledger Stellar.

La constante `BASE_PORCENTAJE` se fija en 100 (`i128`) para permitir aritmética entera exacta sin pérdida por redondeo. Las comisiones se validan en `inicializar` para que su suma no supere el 100 %, rechazando configuraciones inválidas con los errores `ComisionesMuyAltas` (2) y `ComisionesNegativas` (3).

7.3.4. Política de comisiones (5 % organizador, 3 % plataforma)

La política de comisiones del prototipo en testnet aplica un 5 % al organizador y un 3 % a la plataforma sobre cada operación de reventa, con el 92 % restante destinado al vendedor. Estos valores son parámetros del contrato y se establecen en la operación `inicializar`; pueden modificarse por contrato sin requerir cambios en el código fuente. La política no aplica en venta primaria, donde el 100 % se transfiere al organizador.

7.3.5. Errores tipados y eventos on-chain

El contrato define un enum `ErrorContrato` con dieciséis códigos enteros que cubren las condiciones de fallo del ciclo de vida: 1 `YaInicializado`, 2 `ComisionesMuyAltas`, 3 `ComisionesNegativas`, 4 `PrecioInvalido`, 5 `YaEnVenta`, 6 `BoletoUsado`, 7 `NoEnVenta`, 8 `AutoCompra`, 9 `YaUsado`, 10 `BoletoNoEncontrado`, 11 `NoAutorizado`, 12 `VersionInvalida`, 13 `BoletoInvalidado`, 14 `NoInicializado`, 15 `VerificadorYaExiste`, 16 `VerificadorNoEncontrado`.

Los siete eventos on-chain emitidos son: `boleto_creado`, `boleto_listado`, `venta_cancelada`, `boleto_comprado_primario`, `boleto_revendido`, `boleto_redimido` y `boleto_invalidado_evt`. El indexador suscribe a estos eventos y mantiene la coherencia entre el estado on-chain y la base de datos relacional.

7.4. Arquitectura distribuida (3 nodos)

El prototipo se despliega en tres nodos lógicos con responsabilidades diferenciadas. Esta separación permite escalar de forma independiente cada plano y aislar fallos por dominio.

7.4.1. Nodo Web (UI ticketera)

El frontend está implementado en React 18 con Vite, TypeScript, Tailwind CSS y la librería de componentes shadcn/ui (sobre Radix). La integración Web3 utiliza `@stellar/freighter-api` versión 6.0.1 para la firma del comprador y `@stellar/stellar-sdk` versión 14.6.1 para la construcción local de transacciones de validación. La aplicación expone 23 rutas y cubre el flujo completo de e-commerce (catálogo, carrito, checkout, cuenta, mis entradas, mis ventas P2P) y los flujos administrativos (panel administrador y escáner QR de redención). El servidor de desarrollo opera en el puerto 8080.

7.4.2. Nodo API (Node.js)

El backend se implementa en Node.js con Express y TypeScript, sobre Prisma como ORM contra PostgreSQL. Su módulo de autenticación utiliza `bcryptjs` (factor 10) para el hash de contraseñas y `jsonwebtoken` para la emisión de tokens JWT con vigencia de 7 días. La integración con Stellar se realiza mediante `@stellar/stellar-sdk` versión 14.4.1 (versión fijada para garantizar reproducibilidad en el entorno de despliegue).

El backend expone tres familias de endpoints: (i) catálogo público con caché en memoria de TTL 30–60 s para reducir el tiempo de respuesta a Supabase; (ii) flujo de autenticación, carrito y checkout fiat, transaccional y atómico (creación de `orders`, `order_items`, `tickets` y `payments` en una sola operación de base de datos); (iii) integración Soroban, que incluye los endpoints `secure-ticket` (firma server-side de `crear_boleto`), `list-ticket`,

`cancel-listing`, `build-buy-xdr` (construye XDR sin firmar para el comprador), `submit` (recibe el XDR firmado por Freighter y lo retransmite al RPC), y la cadena de coleccionable Stellar Classic (`mint-collectible`, `transfer-collectible`, `build-trust-xdr`, `submit-classic`).

7.4.3. Nodo Datos (PostgreSQL + Indexer)

La persistencia se gestiona en PostgreSQL hospedado en Supabase, con esquema `ticketing` y un `connection_limit` de 5 para soportar el acceso concurrente del API y el indexador. El esquema relacional comprende los modelos del dominio Web2 (`users`, `events`, `organizers`, `venues`, `venue_sections`, `seats`, `event_ticket_types`, `event_seat_inventory`, `carts`, `cart_items`, `seat_holds`, `orders`, `order_items`, `payments`, `tickets`) y los campos Web3 añadidos al modelo `tickets`: `contract_address`, `ticket_root_id`, `version`, `is_for_sale`, `resale_price` (`BigInt` en stroops), `owner_wallet` y `asset_code` (identificador único del coleccionable Stellar Classic). La unicidad on-chain queda reflejada off-chain mediante el índice único (`contract_address`, `ticket_roo`

El indexador es un proceso de larga duración que poll-ea Soroban RPC cada 5 segundos. Mantiene un cursor en la tabla `indexer_state.last_ledger`; si el ledger inicial cae fuera de la ventana de retención del RPC público, parsea el error de rango y ajusta el cursor automáticamente al mínimo permitido. Procesa los siete eventos on-chain con la siguiente semántica:

- `boleto_creado`: crea el registro `tickets` si no existe.
- `boleto_listado`: `is_for_sale=true`.
- `venta_cancelada`: `is_for_sale=false`.
- `boleto_comprado_primario`: si la operación tiene `resale_price` (caso de venta P2P), cancela el registro del vendedor y crea el registro del comprador con `version+1`; en caso contrario actualiza la propiedad in-place.
- `boleto_revendido`: cancela la versión anterior preservando el `resale_price`, crea la nueva versión con verificación de existencia previa para garantizar idempotencia.
- `boleto_redimido`: `status=USED`.
- `boleto_invalidado_evt`: `status=CANCELLED`.

7.5. Coleccionable Stellar Classic (NFT visible en billetera)

Como capa adicional sobre el modelo de boleto Soroban, el sistema emite un activo Stellar Classic asociado a cada boleto, con código único de 12 caracteres (`T` + 11 caracteres hex derivados del UUID del ticket). El emisor del activo es la dirección del organizador, configurada con los flags `AUTH_REVOCABLE` y `AUTH_CLAWBACK_ENABLED` para habilitar la transferencia automática durante la reventa.

El flujo de emisión y transferencia opera mediante cuatro endpoints: (i) `build-trust-xdr` construye una operación `CHANGE_TRUST` con `limit=1` para que el comprador pueda recibir el activo; (ii) `submit-classic` retransmite la operación clásica firmada vía Horizon (las operaciones clásicas no se enrutan por Soroban RPC); (iii) `mint-collectible` firma server-side la operación `PAYMENT` de una unidad del activo desde el organizador hacia el comprador, con

semántica idempotente; (iv) `transfer-collectible` aplica un `CLAWBACK` sobre la dirección del vendedor anterior y un `PAYMENT` hacia el nuevo comprador en una sola transacción firmada por el organizador.

7.6. Indexación y auditoría (eventos on-chain hacia Postgres)

La indexación es el componente que cierra el ciclo de auditoría del sistema. Su función no es replicar el estado on-chain (que ya es la fuente de verdad) sino habilitar consultas operacionales sobre el historial sin pagar el costo de tiempo y dinero de invocar el RPC. Las consultas que el frontend ejecuta de forma intensiva (`GET /api/tickets`, `GET /api/tickets/sold`, listados de mercado secundario) se resuelven contra PostgreSQL con tiempos del orden de la decena de milisegundos, mientras que la verificación de propiedad on-chain se reserva para los puntos críticos del flujo (compra y redención).

7.6.1. Ventana de retención de getEvents y necesidad de ingesta

El método `getEvents` de Soroban RPC retiene un rango limitado de ledgers (en el caso del RPC público de testnet, del orden de algunos miles de ledgers correspondientes a varios días). Para mantener un historial completo durante toda la duración del prototipo y de la evaluación experimental, el indexador persiste cada evento procesado en PostgreSQL en el momento de su ocurrencia, lo que elimina la dependencia del RPC para consultas sobre eventos antiguos.

7.7. Mapeo normativo simulado (Ley 1493/2011 en diseño, solo documental)

La trazabilidad de cada operación de venta primaria y reventa, junto con el precio efectivo registrado on-chain y persistido en `tickets.resale_price`, habilita el cálculo determinístico de la contribución parafiscal del 10 % establecida por la Ley 1493 de 2011. El cálculo se obtiene como una agregación SQL sobre el historial de eventos `boleto_comprado_primario` y `boleto_revendido` cuyo precio supera el umbral legal de 3 UVT. Este mapeo se incluye como referencia académica del diseño y no constituye una declaración de cumplimiento regulatorio del prototipo.

8. Implementación

Esta sección describe la materialización técnica del diseño propuesto en un entorno de prueba sobre Stellar testnet. La implementación se organiza en tres capas: contratos Soroban en Rust, backend Node.js con Express, y frontend web en React. El código se gestiona como monorepo en el repositorio `https://github.com/Tesis-Stellar/stellar-tickets` bajo la ruta `codigo/main_contract/`.

8.1. Implementación de contratos Soroban

Los contratos se desarrollan en Rust contra `soroban-sdk` versión 23, organizados como un Cargo workspace con dos paquetes: `event_contract` y `factory_contract`. El perfil de compilación `release` aplica `opt-level="z"`, *Link-Time Optimization*, eliminación de símbolos y `panic=.ªbort"`, lo que produce módulos WASM con tamaños reducidos compatibles con el target `wasm32v1-none` requerido por Soroban.

El contrato `event_contract` comprende aproximadamente 1 142 líneas de código fuente en `lib.rs` y 709 líneas de pruebas en `test.rs`, con 35 casos de prueba unitarios. El contrato `factory_contract` comprende 431 líneas de código fuente y 245 líneas de pruebas, con 13 casos. El total de pruebas automatizadas sobre los contratos es de 48 casos, ejecutables mediante `cargo test` en el workspace.

8.1.1. NFT Ticket: emisión, propiedad y versionado

El boleto se modela en `event_contract` bajo la estructura `Boleto` y la clave de almacenamiento `ClaveDato::Boleto(u32, u32)`, donde la tupla corresponde a (`ticket_root_id`, `version`). La asignación del `ticket_root_id` se realiza en la función `crear_boleto` mediante el contador `ContadorBoletos`, que se incrementa atómicamente por cada emisión. Cada cambio de propiedad en reventa incrementa la `version` y actualiza el índice `VersionActual(ticket_root_id)`, que constituye el puntero a la versión activa.

Como capa adicional, el sistema emite un activo Stellar Classic por cada boleto con código alfanumérico de 12 caracteres (`T` + 11 caracteres derivados del UUID off-chain). Este activo es visible directamente en la billetera Freighter del comprador, lo que materializa la representación NFT en una superficie reconocible por el usuario final, sin sustituir al modelo Soroban como fuente de verdad.

8.1.2. Marketplace y pagos (XLM vía SAC)

El marketplace se implementa íntegramente en `event_contract`. Las operaciones `listar_boleto` y `cancelar_venta` actualizan el campo `en_venta` y el precio asociado; la operación `comprar_boleto` ejecuta la liquidación en una sola invocación.

El activo de pago utilizado en el prototipo es XLM expuesto vía Stellar Asset Contract (SAC) con dirección de contrato `CDLZFC3SYJYDZT7K67VZ75HPJVIEUVNIXF47ZG2FB2RMQQVU2HHGCYSC` en testnet. La transferencia se realiza mediante invocación al método `transfer` del SAC dentro del mismo *call frame* de `comprar_boleto`, lo que garantiza que el pago, la distribución de comisiones y el cambio de propiedad ocupen una única transacción del ledger.

8.1.3. Redención y control de acceso

La redención se controla mediante el rol verificador. El organizador registra direcciones autorizadas con `agregar_verificador` y las elimina con `remover_verificador`. La operación `redimir_boleto` valida con `es_verificador` que el invocante esté en la lista, marca el boleto con `usado=true` y emite el evento `boleto_redimido`. Esta separación de roles garantiza que la redención no requiera la llave del organizador en la puerta del evento, lo que reduce la superficie de exposición de claves operativas críticas.

En el prototipo, el flujo de redención de la puerta opera sobre la base de datos relacional para minimizar la latencia frente al usuario final (paso de tiempo dominado por la red, del orden de 5 segundos por confirmación on-chain). La invocación de `redimir_boleto` en el ledger se reserva para escenarios de auditoría posterior y queda documentada como decisión de diseño con su trade-off explícito.

8.2. Backend Node.js + Stellar SDK

El backend está implementado en Node.js con Express, TypeScript y Prisma como ORM. El stack se compone de las dependencias `express`, `@prisma/client`, `@stellar/stellar-sdk` (versión fijada en 14.4.1 para reproducibilidad de despliegue), `cors`, `dotenv`, `bcryptjs` y `jsonwebtoken`. La configuración TypeScript utiliza `target` ES2021 y `module` commonjs.

Las variables de entorno requeridas son `DATABASE_URL` (Supabase con `connection_limit=5`), `SOROBAN_RPC_URL`, `HORIZON_URL` (opcional, por defecto el RPC público de testnet), `PORT`, `JWT_SECRET` (obligatoria en producción, con bloqueo de arranque si no está definida), `ORGANIZER_SECRET` y `FACTORY_CONTRACT_ID`.

8.2.1. API (usuarios, eventos, carrito, checkout, transacciones)

La API expone seis familias de endpoints:

- **Catálogo público:** `GET /api/events`, `GET /api/events/featured`, `GET /api/events/:slug`, `GET /api/events/:id/ticket-types`, `GET /api/events/:id/related`. Caché en memoria con TTL entre 30 y 60 segundos para reducir las consultas repetitivas a Supabase.
- **Autenticación:** `POST /api/auth/login`, `POST /api/auth/register` con `bcrypt.hash(10)`, `GET /api/users/me` y `PATCH /api/users/me`. El `authMiddleware` extrae el `userId` del token JWT y lo inyecta en `req.userId`.
- **Carrito:** `GET /api/cart`, `POST /api/cart/items`, `PATCH /api/cart/items/:id`, `DELETE /api/cart/items/:id`, `DELETE /api/cart/clear`. Las operaciones de modificación filtran por `user_id` para impedir acceso cruzado entre usuarios.
- **Checkout fiat:** `POST /api/checkout/preview` y `POST /api/checkout/confirm`. El segundo crea `orders`, `order_items`, `tickets` y `payments` en una transacción Prisma atómica y marca el carrito como `CONVERTED`.
- **Soroban:** `POST /api/transactions/secure-ticket` (firma de `crear_boleto`), `POST /api/transactions/list-ticket`, `POST /api/transactions/cancel-listing`, `POST /api/transaction

(XDR sin firmar), `POST /api/transactions/submit` (retransmisión del XDR firmado por Freighter al RPC).

- **Coleccionable Stellar Classic:** `POST /api/transactions/build-trust-xdr`, `POST /api/transactions/submit-classic`, `POST /api/transactions/mint-collectible`, `POST /api/transactions/transfer-collectible`.
- **Administración (rol ADMIN):** `GET /api/admin/events`, `POST /api/admin/events`, `POST /api/admin/events/:id/deploy` (despliegue de un nuevo contrato a través del factory), `GET /api/admin/contracts`, y `POST /api/admin/scan` (acceso para roles ADMIN y STAFF).

8.2.2. Indexer (ingesta de eventos)

El indexador se implementa como un proceso de larga duración en `src/indexer.ts` que consulta Soroban RPC cada 5 segundos. Mantiene un cursor en la tabla `indexer_state.last_ledger`, con valor inicial de 100 ledgers atrás. Procesa los eventos en chunks de hasta 1 000 ledgers por solicitud y agrupa hasta cinco direcciones de contrato por filtro, conforme al límite del RPC.

Cada uno de los siete eventos (`boleto_creado`, `boleto_listado`, `venta_cancelada`, `boleto_comprado_primario`, `boleto_revendido`, `boleto_redimido`, `boleto_invalidado_evt`) tiene un manejador dedicado con semántica idempotente: la verificación de existencia previa ante `boleto_revendido` evita conflictos por reentrega del evento, y los manejadores P2P preservan el campo `order_item_id` entre versiones para conservar la vinculación con la orden original. Ante errores transitorios el indexador reintenta tras 5 segundos; si el cursor cae fuera de la ventana de retención del RPC, parsea el error y se reposiciona automáticamente al mínimo permitido.

El indexador no se inicia cuando la variable `VERCEL=1` está presente, dado que los entornos serverless no soportan procesos de fondo. Para despliegues sobre Render se utiliza la modalidad de servicio web persistente con el comando `tsx src/server.ts`.

8.2.3. Persistencia PostgreSQL (historial, listados, auditoría)

El esquema relacional se define en Prisma bajo el namespace `ticketing`. Los enumerados del dominio incluyen `user_role` (CUSTOMER, ADMIN, STAFF), `event_status`, `cart_status`, `order_status`, `payment_method` (CARD, PSE, CASHPOINT), `payment_status`, `seat_inventory_status` (AVAILABLE, HELD, SOLD, BLOCKED), `ticket_status` (ACTIVE, USED, CANCELLED, REFUNDED) y `venue_type`.

Los campos Web3 añadidos al modelo `tickets` son `contract_address`, `ticket_root_id`, `version`, `is_for_sale`, `resale_price` (`BigInt`, en stroops), `owner_wallet` y `asset_code` (único). El índice único compuesto (`contract_address`, `ticket_root_id`, `version`) refleja la unicidad on-chain en el plano relacional.

8.3. Frontend web (simulación ticketera)

La aplicación frontend se ejecuta sobre Vite y emplea TanStack Query para gestión de caché HTTP, Framer Motion para transiciones de interfaz y la familia de componentes shadcn/ui sobre Radix. La integración con la billetera del usuario utiliza el SDK `@stellar/freighter-api`

versión 6.0.1, cuya interfaz devuelve objetos con campos `isConnected`, `address` y `signedTxXdr`.

El estado global de la aplicación se gestiona en `AppContext.tsx` y comprende: usuario autenticado y token JWT (persistente en `localStorage.authToken`), carrito, órdenes, entradas adquiridas, ventas P2P completadas y dirección de la billetera vinculada. La función `apiFetch<T>(path, init?)` centraliza todas las llamadas al backend con `Bearer` token automático.

El componente `ConnectWallet.tsx` se renderiza únicamente cuando el usuario está autenticado, lee la dirección de la billetera desde el contexto persistente y solo invoca a Freighter cuando no hay dirección previa, evitando solicitudes redundantes al usuario al navegar entre páginas. El componente `TicketCard.tsx` expone los flujos "Asegurar en Blockchain" (mint del boleto on-chain y emisión del coleccionable Classic) y "Revender NFT" (diálogo modal con precio en COP, vista previa en XLM en tiempo real mediante el hook `useXlmPrice` contra la API pública de CoinGecko, con caché de 5 minutos en `localStorage`).

8.4. Despliegue distribuido (3 nodos)

El despliegue del prototipo se realiza sobre tres nodos diferenciados:

- **Capa de presentación:** frontend desplegado en Vercel con archivo `vercel.json` que configura las reescrituras para el modelo SPA.
- **Capa de servicios:** backend desplegado en Render como servicio web persistente. Esta modalidad es necesaria porque el indexador requiere ejecución continua en segundo plano, condición no soportada por entornos serverless puros.
- **Capa de datos:** PostgreSQL gestionado por Supabase, con el esquema `ticketing` y `connection_limit=5` para soportar el acceso concurrente del API y el indexador.

El despliegue de los contratos sobre Stellar testnet se ejecuta mediante el script `backend/scripts/deploy-contra`
El flujo comprende: (i) carga de keypairs desde `.env.deploy`; (ii) financiación mediante *friendbot*; (iii) carga del módulo WASM al ledger; (iv) despliegue de un contrato `event_contract` por evento mediante el factory; (v) invocación de `inicializar` con las comisiones del 5 % y 3 %; (vi) persistencia de la dirección del contrato en el campo `events.contract_address`. Al cierre de esta tesis hay 11 contratos de evento desplegados en testnet bajo el factory configurado.

8.5. Integración continua

El repositorio incluye un workflow de GitHub Actions en `.github/workflows/soroban.yml` que se ejecuta en cada *push* y cada *pull request* contra la rama `main`. El workflow utiliza `cargo-binstall` para instalar `stellar-cli` sin compilación desde fuentes, ejecuta la compilación por paquete (`-package event_contract`, `-package factory_contract`) para evitar colisiones de símbolos en el target `wasm32v1-none`, ejecuta la suite de pruebas con `cargo test`, y publica el módulo WASM compilado como artefacto del job. Esta automatización constituye la evidencia operacional de la trazabilidad de cambios requerida por el marco metodológico ISO 9001.

8.6. Pruebas (unitarias, integración, antifraude, medición)

La estrategia de pruebas se organiza en cuatro niveles:

- **Pruebas unitarias de contratos:** 48 casos en total (35 sobre `event_contract` y 13 sobre `factory_contract`), ejecutables con `cargo test`. Cubren el ciclo de vida completo del boleto, las precondiciones de cada función, los 16 errores tipados y la doble autenticación del factory.
- **Pruebas de integración:** flujo de extremo a extremo que comprende registro de usuario, navegación del catálogo, compra fiat, "Asegurar en Blockchain", listado de reventa, compra de reventa con firma Freighter, redención por verificador y cancelación administrativa.
- **Pruebas antifraude:** escenarios negativos que validan el rechazo del contrato ante doble redención, doble compra, autocompra (vendedor que intenta comprar su propio boleto, error `AutoCompra` código 8), redención por dirección no autorizada (error `NoAutorizado` código 11), y manipulación del precio en el cliente (la firma del comprador queda invalidada por la verificación on-chain del precio listado).
- **Pruebas de medición:** captura de tiempos y costos sobre testnet para los flujos de emisión, listado, compra de reventa y redención. Las métricas de latencia, comisión por operación y tasa de fallos alimentan la sección de evaluación y resultados.

9. Estándares de ingeniería

9.1. ISO/IEC 27001 (controles como requisitos no funcionales; sin certificación)

En este trabajo, ISO/IEC 27001 se adopta como marco de referencia para definir controles de seguridad de la información a nivel de diseño e implementación. Su aplicación se limita al contexto académico del prototipo y no implica un proceso de certificación formal.

9.2. ISO 9001 (gestión de calidad del SDLC y control documental)

ISO 9001 se utiliza como guía para estructurar prácticas de calidad durante el ciclo de vida del desarrollo de software, con énfasis en trazabilidad de decisiones, consistencia de artefactos y control de cambios.

9.3. Estándares de documentación, versionamiento y trazabilidad

Se establecen criterios para documentación técnica, manejo de versiones y registro de evidencias de pruebas, de manera que el proceso de desarrollo y evaluación del prototipo sea reproducible y auditable en el entorno académico.

10. Evaluación y resultados

10.1. Metodología de medición (latencia E2E, confirmación, tasa de fallos)

La evaluación se estructura con métricas cuantitativas de desempeño y consistencia funcional. Se miden tiempos de envío y confirmación de transacciones, latencia extremo a extremo desde interfaz, y proporción de fallos por tipo de operación.

10.2. Resultados del prototipo (Testnet)

En esta subsección se consolidan resultados de ejecución sobre testnet para los flujos de emisión, listado, compra, transferencia y redención. La presentación de resultados incluye tablas comparativas y análisis por percentiles cuando aplique.

10.3. Evaluación antifraude (doble redención, reventa múltiple, QR estático)

Se prueban escenarios de riesgo priorizados para validar que el sistema rechaza usos inválidos y conserva trazabilidad verificable del ticket a lo largo de su ciclo de vida.

10.4. Discusión comparativa (contra estado del arte)

Los resultados obtenidos se contrastan con referencias del estado del arte para discutir alcances, limitaciones y viabilidad técnica del enfoque basado en contratos inteligentes.

11. Conclusiones

11.1. Conclusiones

Este trabajo estructuró y validó un enfoque de reventa B2B de entradas sobre Stellar, articulando diseño contractual, arquitectura distribuida e instrumentación de métricas. La propuesta permite analizar cómo la combinación de unicidad del ticket, verificación de propiedad y trazabilidad de eventos contribuye a mitigar escenarios de fraude en el mercado secundario.

11.2. Trabajo futuro (mainnet, KYC, integraciones, políticas avanzadas)

Como extensiones de investigación se plantea el despliegue controlado en mainnet, incorporación de mecanismos de identidad/KYC, integración con plataformas reales de ticketing y evaluación de políticas de mercado avanzadas para gestión de especulación y cancelaciones.

A. Anexos

A.1. Manual de usuario

En este anexo se incluirá la guía operativa para los roles del prototipo: administrador de ticketera, vendedor, comprador y verificador.

A.2. Manual de despliegue

En este anexo se documentarán prerequisitos, variables de entorno, pasos de despliegue y validación en testnet.

A.3. Casos de prueba y scripts de medición

En este anexo se consolidarán los casos unitarios, de integración y antifraude, junto con scripts para medición de latencia y tasa de fallos.

A.4. ABI/IDs de contratos y evidencias

En este anexo se registrarán identificadores de contratos, direcciones relevantes y evidencias de ejecución para trazabilidad académica.